

Sharif University of Technology

Department of Electrical Engineering

Digital Electronics

Complex 2x2 Matrix Determinant Calculator

Student Name and ID:

Helia Shakeri

ID: 404200739

Professor:

Dr. Sharifkhani

February 19, 2026

Fall Semester 2025

Contents

1	Verilog Code	2
1.1	ADD16	2
1.2	complexADD16	2
1.3	MULT8	2
1.4	complexMULT8	3
1.5	Determinant	3
2	Synthesis	9
2.1	Design Compiler	9
3	Questa Sim	11
4	Innovus	11
5	Results	12

List of Figures

1	The structure of the determinant module	7
2	The area report	9
3	The power report	10
4	The timing report	10
5	The waveform of the simulated test bench	11
6	The layout of the designed ASIC	12

List of Tables

1	FSM states for determinant calculation (pipelined multiplier/adder)	7
2	Determinant calculation pipeline trace (64-cycle load + 21-cycle compute)	8
3	The design characteristics	12

Abstract

In this report, a complex 2x2 determinant calculator is designed and synthesized. The first section describes the Verilog code for the project and the data flow table, then the next section describes the synthesis process and reports.

1 Verilog Code

1.1 ADD16

The first module is a simple combinational 16-bit adder. The adder could've been further optimized for area but the added latency would've violated the 64-cycle constraint and would not have been worth the meager decrease in area.

```
module ADD16 (  
    input  signed [15:0] a,  
    input  signed [15:0] b,  
    output signed [15:0] sum  
);  
  
    assign sum = a + b;  
  
endmodule
```

1.2 complexADD16

Using the ADD16 module, an adder for 2 16-bit complex numbers was designed. To decrease the area, only one ADD16 was used which made the module sequential and needed an FSM that took 4 cycles. Considering the fact that complex adding is only used once in the determinant calculation process, the latency overhead is acceptable to reduce the area.

But when the code was synthesized later and compared to a combinational adder that used 2 ADD16 modules, the combinational one had less area, probably due to the controller FSM overhead.

Finally, the code was written without the complexMULT8 module that will be introduced later and it became clear that using the simple ADD16 is more area-efficient than using this module, so it was discarded.

1.3 MULT8

This module is a simple 8-bit by 8-bit combinational multiplier.

In order to properly reduce the area, one idea is to make the multiplier entirely sequential and use resource sharing by implementing a Booth structure. The problem with Booth is that it takes 8 cycles to calculate a 8x8 multiplication, and considering the determinant needs 8 multiplications, $8 \times 8 = 64$ which would violate the latency constraint.

To reduce the Booth latency, a radix-4 Booth structure can be used, which takes 4 to 5 cycles to calculate 8x8 multiplications. As witnessed in the complexADD16 module, the FSM and controller overhead would probably negate most of the benefits of the sequential design and make the high latency too much of a cost for too little gain.

```
module MULT8 (
input  signed [7:0] a,
input  signed [7:0] b,
output signed [15:0] result
);

assign result = a * b;

endmodule
```

1.4 complexMULT8

This module uses one MULT8 and one ADD16 module to compute the multiplication of 2 complex 8-bit numbers. The FSM necessary to control this module has 7 states from IDLE to DONE, computes 4 multiplications in these states and 2 additions, and waits for one cycle after outputs are produced to raise the valid flag to remove any race conditions.

But when the code was synthesized with and without this module, it became clear that the code using this module took up more space than the one without so this module was discarded too.

1.5 Determinant

The first determinant module used one complexAdd16 and one complexMULT8 module to reduce the area. It must be noted that this determinant code cannot handle continuous inputs and can only accept the next input once the valid flag is raised.

Another version of this module was written that used one ADD16 and one MULT8 unit with a more complicated FSM with 17 states.

These two modules were both synthesized and compiled to optimize the area. The first module yielded the minimum area of 5911 and the second one 4054. Turns out that the complexMULT and complexADD modules complicated the design too much and using the simpler modules lead to less area. The code for the second module is shown here and fig. 1 shows the blocks and overall structure of the resource-sharing code.

```
module determinant (
input  wire      clk,
input  wire      rst,
input  wire      S,
input  wire      F,
output reg signed [15:0] det_real,
output reg signed [15:0] det_imag,
```

```

output reg          valid
);

reg signed [7:0] a11_r, a11_i;
reg signed [7:0] a12_r, a12_i;
reg signed [7:0] a21_r, a21_i;
reg signed [7:0] a22_r, a22_i;

reg [2:0] word_cnt;
reg [2:0] bit_cnt;
reg [7:0] shift_reg;

reg signed [7:0] mult_a, mult_b;
wire signed [15:0] mult_out;

MULT8 u_mult (
.a(mult_a),
.b(mult_b),
.result(mult_out)
);

// ADD16
reg signed [15:0] add_a, add_b;
wire signed [15:0] add_out;

ADD16 u_add (
.a(add_a),
.b(add_b),
.sum(add_out)
);

reg [5:0] state;

parameter IDLE=0, LOAD=1,

M1_1=2, M1_2=3, M1_3=4, M1_4=5,
M1_R_SET=6, M1_R_GET=7,
M1_I_SET=8, M1_I_GET=9,

M2_1=10, M2_2=11, M2_3=12, M2_4=13,
M2_R_SET=14, M2_R_GET=15,
M2_I_SET=16, M2_I_GET=17,

DET_R_SET=18, DET_R_GET=19,
DET_I_SET=20, DET_I_GET=21,

```

```

DONE=22;

reg signed [15:0] t1,t2,t3,t4;
reg signed [15:0] p1_r,p1_i;
reg signed [15:0] p2_r,p2_i;

always @(posedge clk or posedge rst) begin
if (rst) begin
state <= IDLE;
valid <= 0;
word_cnt <= 0;
bit_cnt <= 0;
det_real <= 0;
det_imag <= 0;
end
else begin

valid <= 0;

case(state)

IDLE:
if (F) begin
state <= LOAD;
word_cnt <= 0;
bit_cnt <= 0;
end

LOAD: begin
shift_reg <= {shift_reg[6:0], S};
bit_cnt <= bit_cnt + 1;

if (bit_cnt == 7) begin
case(word_cnt)
0: a11_r <= {shift_reg[6:0], S};
1: a11_i <= {shift_reg[6:0], S};
2: a12_r <= {shift_reg[6:0], S};
3: a12_i <= {shift_reg[6:0], S};
4: a21_r <= {shift_reg[6:0], S};
5: a21_i <= {shift_reg[6:0], S};
6: a22_r <= {shift_reg[6:0], S};
7: a22_i <= {shift_reg[6:0], S};
endcase
word_cnt <= word_cnt + 1;
bit_cnt <= 0;

```

```

if (word_cnt == 7)
state <= M1_1;
end
end

M1_1: begin mult_a<=a11_r; mult_b<=a22_r; state<=M1_2; end
M1_2: begin t1<=mult_out; mult_a<=a11_i; mult_b<=a22_i; state<=M1_3; end
M1_3: begin t2<=mult_out; mult_a<=a11_r; mult_b<=a22_i; state<=M1_4; end
M1_4: begin t3<=mult_out; mult_a<=a11_i; mult_b<=a22_r; state<=M1_R_SET; end

M1_R_SET: begin t4<=mult_out; add_a<=t1; add_b<=-t2; state<=M1_R_GET; end
M1_R_GET: begin p1_r<=add_out; state<=M1_I_SET; end

M1_I_SET: begin add_a<=t3; add_b<=t4; state<=M1_I_GET; end
M1_I_GET: begin p1_i<=add_out; state<=M2_1; end

M2_1: begin mult_a<=a12_r; mult_b<=a21_r; state<=M2_2; end
M2_2: begin t1<=mult_out; mult_a<=a12_i; mult_b<=a21_i; state<=M2_3; end
M2_3: begin t2<=mult_out; mult_a<=a12_r; mult_b<=a21_i; state<=M2_4; end
M2_4: begin t3<=mult_out; mult_a<=a12_i; mult_b<=a21_r; state<=M2_R_SET; end

M2_R_SET: begin t4<=mult_out; add_a<=t1; add_b<=-t2; state<=M2_R_GET; end
M2_R_GET: begin p2_r<=add_out; state<=M2_I_SET; end

M2_I_SET: begin add_a<=t3; add_b<=t4; state<=M2_I_GET; end
M2_I_GET: begin p2_i<=add_out; state<=DET_R_SET; end

DET_R_SET: begin add_a<=p1_r; add_b<=-p2_r; state<=DET_R_GET; end
DET_R_GET: begin det_real<=add_out; state<=DET_I_SET; end

DET_I_SET: begin add_a<=p1_i; add_b<=-p2_i; state<=DET_I_GET; end
DET_I_GET: begin det_imag<=add_out; state<=DONE; end

DONE: begin valid<=1; state<=IDLE; end

endcase
end
end

endmodule

```

Table 1 shows the workings of the FSM, while table 2 shows the timing schedule.

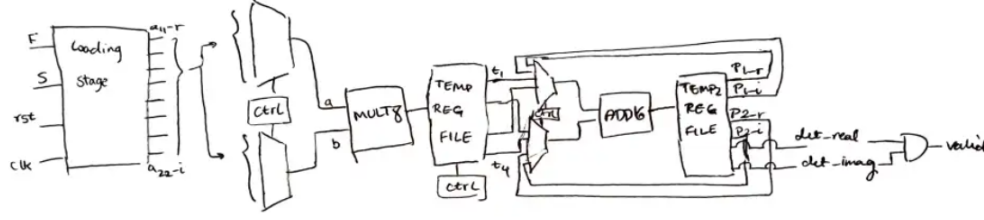


Figure 1: The structure of the determinant module

FSM State	Operation	MULT8 Inputs	ADD16 Inputs	Temp / Output Updated
IDLE	Wait for F	—	—	—
LOAD	Shift in S into <code>shift_reg</code>	—	—	<code>a11_r...a22_i</code>
First Complex Multiplication: $p1 = a11 \times a22$				
M1_1	Start mult: $a11_r \times a22_r$	<code>a11_r, a22_r</code>	—	—
M1_2	Get <code>t1</code> , start $a11_i \times a22_i$	<code>a11_i, a22_i</code>	—	<code>t1</code>
M1_3	Get <code>t2</code> , start $a11_r \times a22_i$	<code>a11_r, a22_i</code>	—	<code>t2</code>
M1_4	Get <code>t3</code> , start $a11_i \times a22_r$	<code>a11_i, a22_r</code>	—	<code>t3</code>
M1_R_SET	Get <code>t4</code> , setup real add ($t1 - t2$)	—	<code>t1, -t2</code>	<code>t4</code>
M1_R_GET	Get <code>p1_r</code> from adder	—	—	<code>p1_r</code>
M1_I_SET	Setup imag add ($t3 + t4$)	—	<code>t3, t4</code>	—
M1_I_GET	Get <code>p1_i</code> from adder	—	—	<code>p1_i</code>
Second Complex Multiplication: $p2 = a12 \times a21$				
M2_1	Start mult: $a12_r \times a21_r$	<code>a12_r, a21_r</code>	—	—
M2_2	Get <code>t1</code> , start $a12_i \times a21_i$	<code>a12_i, a21_i</code>	—	<code>t1</code>
M2_3	Get <code>t2</code> , start $a12_r \times a21_i$	<code>a12_r, a21_i</code>	—	<code>t2</code>
M2_4	Get <code>t3</code> , start $a12_i \times a21_r$	<code>a12_i, a21_r</code>	—	<code>t3</code>
M2_R_SET	Get <code>t4</code> , setup real add ($t1 - t2$)	—	<code>t1, -t2</code>	<code>t4</code>
M2_R_GET	Get <code>p2_r</code> from adder	—	—	<code>p2_r</code>
M2_I_SET	Setup imag add ($t3 + t4$)	—	<code>t3, t4</code>	—
M2_I_GET	Get <code>p2_i</code> from adder	—	—	<code>p2_i</code>
Determinant: $det = p1 - p2$				
DET_R_SET	Setup real subtract ($p1_r - p2_r$)	—	<code>p1_r, -p2_r</code>	—
DET_R_GET	Get <code>det_real</code> from adder	—	—	<code>det_real</code>
DET_I_SET	Setup imag subtract ($p1_i - p2_i$)	—	<code>p1_i, -p2_i</code>	—
DET_I_GET	Get <code>det_imag</code> from adder	—	—	<code>det_imag</code>
DONE	Output valid	—	—	<code>valid = 1</code>

Table 1: FSM states for determinant calculation (pipelined multiplier/adder)

Cycle	FSM State	Operation	t1	t2	t3	t4	p1_r	p1_i	p2_r	p2_i	det_real	det_imag	valid
0	IDLE	Wait for F	—	—	—	—	—	—	—	—	—	—	0
1-64	LOAD	Serial load (8 words $\times$ 8 bits)	—	—	—	—	—	—	—	—	—	—	0
65	M1_1	Start a11_r $\times$ a22_r	—	—	—	—	—	—	—	—	—	—	0
66	M1_2	Get t1, start a11_i $\times$ a22_i	t1	—	—	—	—	—	—	—	—	—	0
67	M1_3	Get t2, start a11_r $\times$ a22_i	t1	t2	—	—	—	—	—	—	—	—	0
68	M1_4	Get t3, start a11_i $\times$ a22_r	t1	t2	t3	—	—	—	—	—	—	—	0
69	M1_R_SET	Get t4, setup add (t1 - t2)	t1	t2	t3	t4	—	—	—	—	—	—	0
70	M1_R_GET	Get p1_r	t1	t2	t3	t4	p1_r	—	—	—	—	—	0
71	M1_I_SET	Setup add (t3 + t4)	t1	t2	t3	t4	p1_r	—	—	—	—	—	0
72	M1_I_GET	Get p1_i	t1	t2	t3	t4	p1_r	p1_i	—	—	—	—	0
73	M2_1	Start a12_r $\times$ a21_r	t1	t2	t3	t4	p1_r	p1_i	—	—	—	—	0
74	M2_2	Get t1, start a12_i $\times$ a21_i	t1	t2	t3	t4	p1_r	p1_i	—	—	—	—	0
75	M2_3	Get t2, start a12_r $\times$ a21_i	t1	t2	t3	t4	p1_r	p1_i	—	—	—	—	0
76	M2_4	Get t3, start a12_i $\times$ a21_r	t1	t2	t3	t4	p1_r	p1_i	—	—	—	—	0
77	M2_R_SET	Get t4, setup add (t1 - t2)	t1	t2	t3	t4	p1_r	p1_i	—	—	—	—	0
78	M2_R_GET	Get p2_r	t1	t2	t3	t4	p1_r	p1_i	p2_r	—	—	—	0
79	M2_I_SET	Setup add (t3 + t4)	t1	t2	t3	t4	p1_r	p1_i	p2_r	—	—	—	0
80	M2_I_GET	Get p2_i	t1	t2	t3	t4	p1_r	p1_i	p2_r	p2_i	—	—	0
81	DET_R_SET	Setup add (p1_r - p2_r)	—	—	—	—	p1_r	p1_i	p2_r	p2_i	—	—	0
82	DET_R_GET	Get det_real	—	—	—	—	p1_r	p1_i	p2_r	p2_i	det_real	—	0
83	DET_I_SET	Setup add (p1_i - p2_i)	—	—	—	—	p1_r	p1_i	p2_r	p2_i	det_real	—	0
84	DET_I_GET	Get det_imag	—	—	—	—	p1_r	p1_i	p2_r	p2_i	det_real	det_imag	0
85	DONE	Output valid	—	—	—	—	p1_r	p1_i	p2_r	p2_i	det_real	det_imag	1

Table 2: Determinant calculation pipeline trace (64-cycle load + 21-cycle compute)

2 Synthesis

2.1 Design Compiler

The Verilog code of the modules was combined into one module along with a test bench and copied to the folders created in the Cadence folder.

Then in the Design Compiler, the code was added, analyzed and elaborated. Then the compile process began. First the Compile Design option was attempted with high area effort. Then using the Compile Ultra option, every possible combination of selections was tried (the retime and gate clock options proved the most helpful) and the area reached 3451.

Then the clock was set to 1.55075 to meet the slack requirements with no overhead.

Afterwards, the area, power, and timing reports were created and saved in the Report folder. Fig. 2, 3, and 4 show screenshots of the reports mentioned.

The timing report shows that the critical path length was 1.55074 and the maximum clock frequency is 644MHz.

```
Number of ports:          125
Number of nets:           923
Number of cells:          813
Number of combinational cells: 478
Number of sequential cells: 313
Number of macros/black boxes: 0
Number of buf/inv:        88
Number of references:     64

Combinational area:       1271.160015
Buf/Inv area:             95.760004
Noncombinational area:    2180.520041
Macro/Black Box area:     0.000000
Net Interconnect area:    undefined (Wire load has zero net area)

Total cell area:          3451.680056
Total area:               undefined
1
```

Figure 2: The area report

Finally, the netlist file was saved and the SDF file was created.

```

Global Operating Voltage = 1.32
Power-specific unit information :
  Voltage Units = 1V
  Capacitance Units = 1.000000pf
  Time Units = 1ns
  Dynamic Power Units = 1mW      (derived from V,C,T units)
  Leakage Power Units = 1nW

Cell Internal Power   = 26.4863 uW   (68%)
Net Switching Power   = 12.7266 uW   (32%)
-----
Total Dynamic Power   = 39.2130 uW   (100%)

Cell Leakage Power    = 168.5103 nW

Information: report_power power group summary does not include estimated clock tree power. (PWR-789)

```

Power Group	Internal Power	Switching Power	Leakage Power	Total Power	(%)	Attrs
io_pad	0.0000	0.0000	0.0000	0.0000	(0.00%)	
memory	0.0000	0.0000	0.0000	0.0000	(0.00%)	
black_box	0.0000	0.0000	0.0000	0.0000	(0.00%)	
clock_network	0.0000	0.0000	0.0000	0.0000	(0.00%)	
register	0.0000	0.0000	0.0000	0.0000	(0.00%)	
sequential	2.3667e-02	1.1594e-03	117.3068	2.4944e-02	(63.34%)	
combinational	2.8191e-03	1.1567e-02	51.1836	1.4438e-02	(36.66%)	
Total	2.6486e-02 mW	1.2727e-02 mW	168.4904 nW	3.9381e-02 mW		

Figure 3: The power report

Point	Incr	Path
clock clk (rise edge)	0.00000	0.00000
clock network delay (ideal)	0.00000	0.00000
mult_a_reg[3]/CP (DFQD1)	0.00000	0.00000 r
mult_a_reg[3]/Q (DFQD1)	0.10225	0.10225 f
U3047/ZN (INVD0)	0.08222	0.18447 r
U3057/ZN (AOI22D0)	0.06841	0.25289 f
U3058/ZN (AOI22D0)	0.14064	0.39352 r
U3064/ZN (INVD0)	0.04942	0.44294 f
U3104/ZN (AOI22D0)	0.07319	0.51613 r
U3113/S (FA1D0)	0.10670	0.62283 f
U3114/ZN (INVD0)	0.02023	0.64306 r
U3125/S (FA1D0)	0.06031	0.70337 f
U3126/ZN (INVD0)	0.02819	0.73156 r
U3208/CO (FA1D0)	0.09357	0.82513 r
U3209/CO (FA1D0)	0.09405	0.91918 r
U3210/CO (FA1D0)	0.09405	1.01323 r
U3211/CO (FA1D0)	0.09404	1.10727 r
U3212/CO (FA1D0)	0.09404	1.20131 r
U3213/CO (FA1D0)	0.09405	1.29536 r
U3214/CO (FA1D0)	0.09404	1.38940 r
U3215/CO (FA1D0)	0.08555	1.47496 r
U3216/S (FA1D0)	0.07578	1.55074 r
t1_reg[14]/D (DFQD1)	0.00000	1.55074 r
data arrival time		1.55074
clock clk (rise edge)	1.57044	1.57044
clock network delay (ideal)	0.00000	1.57044
t1_reg[14]/CP (DFQD1)	0.00000	1.57044 r
library setup time	-0.01969	1.55075
data required time		1.55075
data required time		1.55075
data arrival time		-1.55074
slack (MET)		0.00001

Figure 4: The timing report

3 Questa Sim

Using Questa Sim, the Verilog code was simulated with the SDF file produced by the Design Compiler. Fig. 5 shows the waveform of the test bench example, which was an identity matrix. The cursors show how long it takes for the code to raise the valid flag since the moment the F flag is raised and the frame starts. Considering that the loading stage takes 64 cycles itself, the calculations only take 25 cycles and have a 25 cycle latency. The throughput, considering the fact that the input has to wait for the valid flag to be raised and calculations to be over before the next one, would be 1 result every 80 cycles.

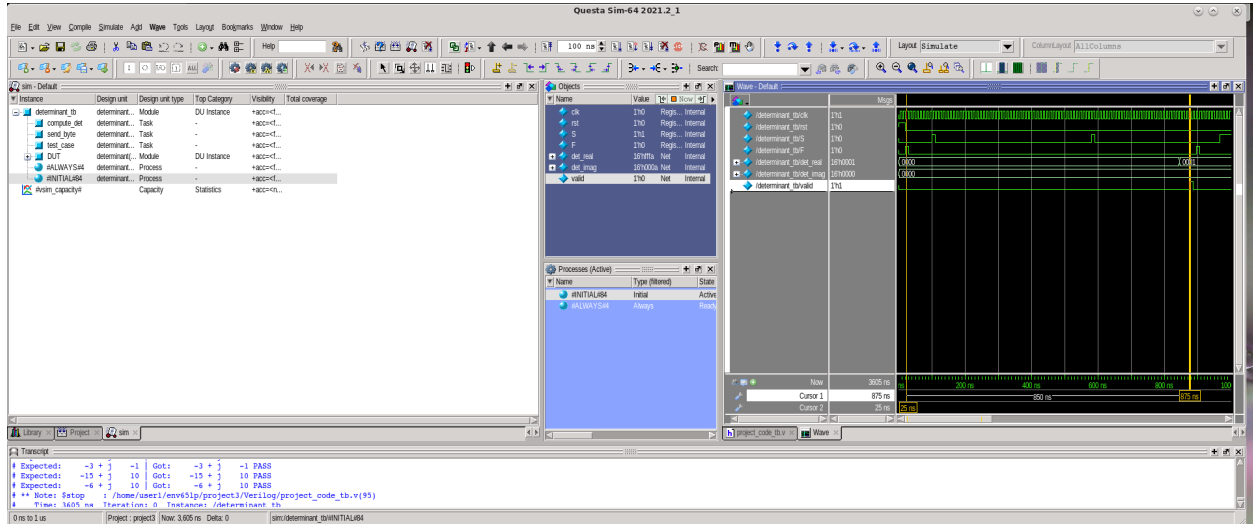


Figure 5: The waveform of the simulated test bench

4 Innovus

In this application, the netlist file produced from Design Compiler was imported. Then the VDD and VSS rings and stripes were designed. The number of striped was chosen to be 10 for each net considering the large area of the design. After standard cells were placed and the routing was done, the ruler was used to find the final area of the ASIC, as seen in fig. 6. Considering the width and height, the final area is calculated to be 5505.

Afterwards, the GDSII and DEF files were saved.

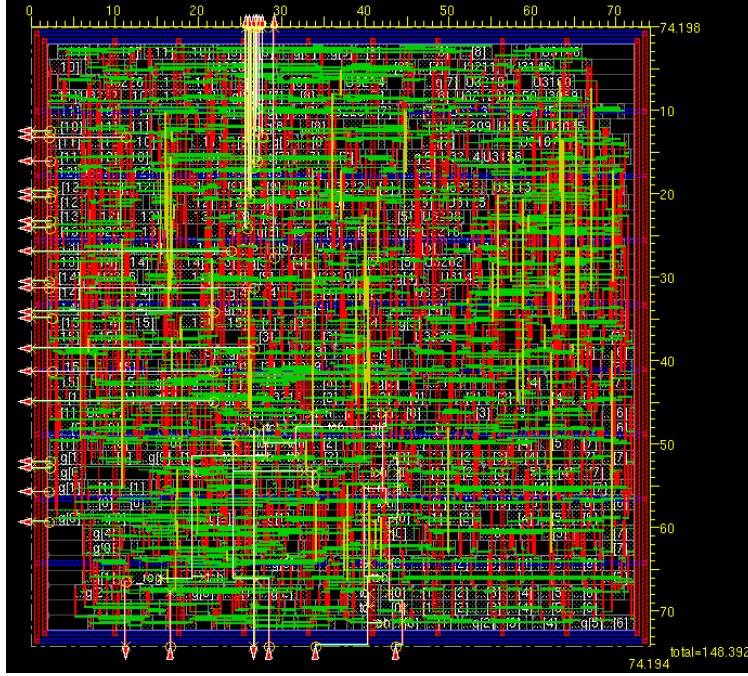


Figure 6: The layout of the designed ASIC

5 Results

Latency	Throughput	Area	Max. Clock Freq.	Power
25 (calculation)	1/79	3451 (5505) μm^2	644MHz	39.38 μW

Table 3: The design characteristics

*The characteristics were written with the assumption that the time unit in Design Compiler is ns and the length unit in Innovus and Design Compiler is μm .

The Verilog codes, Design Compiler reports, and the layout are in the src folder in their respective folders.